

Техническое задание для разработчика

Дашборд «Точки роста» — BASTION GROUP

Разработчик Б

Экраны: Эффективность, Механики

ID-5817, AppCode: ID_5817_DASHBOARD

Дата подготовки: 18.07.2026

1. О проекте

Веб-дашборд «Точки роста» — SPA-приложение для автоцентров BASTION GROUP. Показывает KPI и аналитику по оборотам, мастерам, механикам и автопарку клиентов за выбранный период.

Мультибазное приложение: пользователь может иметь доступ к нескольким автоцентрам (базам данных), между ними переключается глобальный фильтр в шапке.

Стек

- Frontend: React 18, MobX (стейт-менеджмент), Recharts (графики)
- Стилизация: CSS-модули (импорт модульных стилей в компонентах), без CSS-in-JS
- Архитектура: FSD (Feature-Sliced Design)
- Backend: готовый HTTP API поверх SQL Server (PHP-методы), фронтенду доступен только по HTTP — прямого доступа к базе нет

Базовые константы API

Параметр	Значение
Base URL (тест)	https://api-t3.basgroup.ru
AppCode	ID_5817_DASHBOARD
DB_GUID (тест)	3a166f27-77f9-4a0c-a732-84645966637d

Все запросы к API — GET, формат ответа — JSON. Реализация авторизации и HTTP-клиента уже готова централизованно — см. раздел 3.

2. Как запустить проект

- Установить Node.js LTS (nodejs.org), если ещё нет
- Склонировать репозиторий, открыть папку в VS Code
- В терминале выполнить установку зависимостей и запуск:

```
npm install npm run dev -- --host
```

Открыть адрес из вывода терминала (обычно `http://localhost:5173/`).

Важно: DB_GUID передаётся в адресной строке после хэша (роутинг hash-based), без него логин не работает:

```
http://localhost:5173/#/login?DB_GUID=3a166f27-77f9-4a0c-a732-84645966637d
```

3. Архитектура и что уже готово

Структура проекта построена по FSD (Feature-Sliced Design):

```
src/ └─ app/      — точка входа, роутинг, тема (готово, лид) └─ pages/    —
страницы-экраны — ЗДЕСЬ ваша основная работа └─ widgets/  — крупные блоки
интерфейса (шапка, KPI-блок и т.п.) └─ features/ — действия пользователя (смена
периода, drill-through) └─ entities/ — сущности и MobX-сторы (user, dashboard) —
готово └─ shared/  — api, config, lib (утилиты), ui (базовые компоненты)
```

Правило зависимостей FSD: pages может использовать widgets, features, entities, shared. Нельзя тянуть импорт из pages в entities напрямую в обход паттерна, и нельзя импортировать одну page из другой — общий код выносится в widgets/shared.

Алиас `@` = `src/`, используйте его вместо длинных относительных путей:

```
import { dashboardStore } from '@entities/dashboard/model/dashboardStore'
```

Что уже реализовано централизованно (не дублировать)

- `shared/api/` — HTTP-клиент, работа с сессией (SESSIONID)
- `shared/config/` — константы, разбор DB_GUID из адресной строки
- `entities/user/model/authStore.js` — авторизация, текущий автоцентр (`authStore.currentFirm`), `authStore.dbIndex`
- `entities/dashboard/model/periodsStore.js` и `dashboardStore.js` — вся работа с периодами и данными дашборда (подробности — раздел 4)
- `app/` — роутинг (HashRouter), защита маршрутов (редирект на `/login` без сессии)
- `app/styles/theme.css` — единый файл цветовых переменных: тёмная и светлая тема. Все цвета в CSS-модулях — только через `var(--color-surface)`, `var(--color-accent)` и т.п., хардкод хекс-кодов запрещён.

Конвенции кода

- Стили — CSS-модули, файл `ИмяКомпонента.module.css` рядом с компонентом
- Компоненты, читающие MobX-сторы, оборачивать в `observer` из `mobx-react-lite`
- Форматирование — Prettier настроен, форматирование по сохранению должно работать автоматически
- Числа — формат `ru-RU`, пробел как разделитель тысяч, `₽` без копеек в KPI
- `null` в данных показывать как «—», не приводить к 0 (подробности в разделе 4)

Общие UI-компоненты (shared/ui)

Карточка KPI, обёртки над `Recharts`, таблица, модалка `drill-through`, состояния `loading/empty/error` — готовятся централизованно. Прежде чем писать свой компонент с нуля, проверьте `shared/ui`. Если экран нужно делать раньше, чем компонент готов — верстайте временную версию тем же набором пропсов, чтобы замена была быстрой.

Если для экрана не хватает геттера в dashboardStore — не редактируйте общий файл сами, если не уверены: опишите, что нужно, лиду.

4. Данные и API — как этим пользоваться

Все запросы к API уже реализованы централизованно. Разработчикам экранов не нужно писать fetch/HTTP-запросы самим — только читать готовые MobX-сторы и вызывать их методы.

4.1. Глобальные фильтры (уже подключены в шапке)

- authStore.currentFirm — текущий автоцентр ({ FIRM_SHORT_NAME, ... })
- periodsStore.periodOptions — список периодов для выпадающего списка: [{ value: 202606, label: 'Июнь 2026' }, ...]
- periodsStore.selectedPeriodYm — текущий выбранный период (число, например 202606)
- periodsStore.setSelectedPeriod(periodYm) — сменить период

Важно: dashboardStore сам автоматически перезагружает данные при смене периода или автоцентра — экранам не нужно вручную дёргать загрузку, только читать dashboardStore.*.

4.2. dashboardStore — геттеры

Геттер	Что возвращает
isLoading	идёт загрузка данных за период
error	текст ошибки или null
kpi	KPI текущего периода (объект)
kpiPrev	KPI предыдущего периода (для расчёта Δ)
monthly	12 месяцев, уже развёрнуто от старого к новому
mastersFiltered	мастера, уже отфильтровано MASTER_ID <> -1
mastersPrev	мастера за пред. период (для Δ Т-фактора, джойн по MASTER_ID)
mechanicsFiltered	механики, уже отфильтровано MECHANIC_ID <> -1
marks	марки авто
models	модели авто
yearsFiltered	года выпуска, уже отфильтровано MANUFACTURE_YEAR > 0

Пример использования на странице:

```
import { observer } from 'mobx-react-lite' import { dashboardStore } from
'@/entities/dashboard/model/dashboardStore' export const MastersPage =
observer(function MastersPage() { if (dashboardStore.isLoading) return
<p>Загрузка...</p> if (dashboardStore.error) return
<p>{dashboardStore.error}</p> return ( <div>
{dashboardStore.mastersFiltered.map((m) => ( <div key={m.MASTER_ID}
>{m.MASTER_NAME}: {m.TURNOVER} ₽</div> ))) </div> ) })
```

4.3. Поля данных по разделам ответа

kpi / kpiPrev:

```
PERIOD_YM, TURNOVER, ACCOUNT_COUNT, AVG_CASH, T_FACTOR, ACCURACY,
LABOR_TIME, AVG_SERVICE_TIME, WASTED_TIME_MIN, ARTICLE_WORK_RATIO,
SUMMA_WORK, SUMMA_ARTICLE, RECOMMENDED_SUMM, MECHANIC_COUNT
```

monthly[]: PERIOD_YM, MONTH_LABEL, TURNOVER, ACCOUNT_COUNT, AVG_CASH,
ARTICLE_WORK_RATIO — MONTH_LABEL уже отформатирован сервером, но на английском
("Jun.26"). Для русской подписи используйте formatPeriodLabel из @/shared/lib/periodFormat.

mastersFiltered[] / mastersPrev[]:

```
PERIOD_YM, MASTER_ID, MASTER_NAME, ACCOUNT_COUNT, TURNOVER,
TURNOVER_SHARE, AVG_CASH, ARTICLE_WORK_RATIO, LABOR_TIME, T_FACTOR,
WASTED_TIME_MIN, ACCURACY, SUMMA_WORK, SUMMA_ARTICLE,
RECOMMENDED_SUMM
```

mechanicsFiltered[]:

```
PERIOD_YM, MECHANIC_ID, MECHANIC_NAME, ACCOUNT_COUNT, TURNOVER,
AVG_CASH, ARTICLE_WORK_RATIO, AVG_SERVICE_TIME, LABOR_TIME, T_FACTOR,
WASTED_TIME_MIN, ACCURACY, SUMMA_WORK, SUMMA_ARTICLE,
RECOMMENDED_SUMM
```

△ **В ответе нет mechanicsPrev** (в отличие от mastersPrev). Если нужна Δ для механиков — источник пока не согласован, не выдумывайте сами, сообщите лиду.

marks[]:

```
PERIOD_YM, MARK_ID, MARK_NAME, ACCOUNT_COUNT, TURNOVER,
TURNOVER_SHARE, AVG_CASH, ARTICLE_WORK_RATIO, RECOMMENDED_SUMM,
AVG_SERVICE_TIME, LABOR_TIME, T_FACTOR, WASTED_TIME_MIN, ACCURACY,
SUMMA_WORK, SUMMA_ARTICLE
```

models[] (уже, чем marks — без T_FACTOR/ACCURACY/WASTED_TIME_MIN):

```
PERIOD_YM, MODEL_ID, MODEL_NAME, MARK_ID, MARK_NAME,
ACCOUNT_COUNT, TURNOVER, AVG_CASH, ARTICLE_WORK_RATIO,
RECOMMENDED_SUMM, SUMMA_WORK, SUMMA_ARTICLE
```

yearsFiltered[] (тоже без T_FACTOR/ACCURACY/WASTED_TIME_MIN):

```
PERIOD_YM, MANUFACTURE_YEAR, ACCOUNT_COUNT, TURNOVER, AVG_CASH,
ARTICLE_WORK_RATIO, SUMMA_WORK, SUMMA_ARTICLE
```

4.4. Подводные камни при выводе данных

- ID-заглушки -1 уже отфильтрованы в *Filtered-геттерах; для нефильтрованных данных берите dashboardStore.data напрямую и решайте сами
- null в числовых полях — не приводить к 0, показывать «—» (часто встречается в LABOR_TIME, T_FACTOR, AVG_SERVICE_TIME, RECOMMENDED_SUMM у записей с маленьким ACCOUNT_COUNT)
- Лишние пробелы в именах (например «Смурыгин Евгений Михайлович») — делать .trim() и схлопывать двойные пробелы перед выводом
- Числа форматировать в ru-RU, пробел как разделитель тысяч, ₽ без копеек в KPI

4.5. Drill-through (список 3Н по клику)

```
import { getAccountList } from '@entities/dashboard/api/dashboardApi' import
{ authStore } from '@entities/user/model/authStore' import { periodsStore } from
'@entities/dashboard/model/periodsStore' const accounts = await
getAccountList( periodsStore.selectedPeriodYm, authStore.dbIndex, { filterString:
'MASTER_ID=?', filterParam: [masterId] } )
```

⚠ **Метод ещё не проверен на реальном стенде** — структура ответа может слегка отличаться от ожидаемой. Если увидите неожиданный формат — сообщите лиду, не подгоняйте разметку под догадки.

4.6. Открытые вопросы (пока без ответа)

- Источник Δ Т-фактора для механиков (нет `mechanicsPrev`)
- Точная структура ответа `getAccountList (drill-through)` — не подтверждена практикой
- Права доступа (`SpecialRequests/UserRight`) — опционально по ТЗ, делаем, только если останется время

Если ваш экран упирается в один из этих пунктов — не блокируйтесь: сделайте вёрстку и логику на моках/заглушках, пометьте `TODO` в коде и сообщите лиду.

5. Общие требования ко всем экранам

- Числа — `ru-RU`, пробел между тысячами, `₽` без копеек в KPI
- Состояния: `loading` (скелетон), `empty` («Нет данных за период»), `error` (сообщение + повтор) — используйте компоненты из `shared/ui`, когда появятся
- Тема — только через переменные `var(--color-*)` из `app/styles/theme.css`
- Адаптив — `desktop-first`, но обязательна мобильная вёрстка

Референс дизайна: <https://tochki-rosta.pplx.app/#/> — по визуалу ориентируйтесь на него, а данные и поля берите из этого документа (они сверены с реальным API).

6. Задачи: экраны «Эффективность», «Механики»

6.1. Экран «Эффективность» — маршрут `/efficiency` — `src/pages/efficiency/`

Данные: `dashboardStore.kpi`, `dashboardStore.mastersFiltered`, `dashboardStore.marks`.

KPI (4 шт.)

Метрика	Поле	Ед.	Drill-down	Tooltip
Т-фактор	<code>T_FACTOR</code>	%	→ график по мастерам ниже	Загрузка мощностей
Потери на ЗН	<code>WASTED_TIME_MIN</code>	мин	→ график потерь по мастерам	Простой на ЗН, меньше — лучше
Аккуратность	<code>ACCURACY</code>	%	→ таблица по маркам	Качество исполнения
Выработка	<code>LABOR_TIME</code>	нч	→ «Механики»	Суммарные нормо-часы

Графики (2 шт.) — горизонтальные столбчатые, данные из `mastersFiltered`

График	X	Y	Drill-down	Drill-through	Tooltip
Т-фактор по мастерам	<code>T_FACTOR</code>	<code>MASTER_NAME</code>	клик → «Мастера» с выделением	список ЗН мастера	Мастер: Т-фактор %
Потери на ЗН по мастерам	<code>WASTED_TIME_MIN</code>	<code>MASTER_NAME</code>	то же	ЗН мастера с потерями	Мастер: потери мин

Переход на «Мастера» с выделением: `navigate('/masters', { state: { highlightMasterId: id } })`, страница «Мастера» читает `location.state?.highlightMasterId`. Согласовать точное название поля с разработчиком А.

Таблица «Эффективность по маркам» — `dashboardStore.marks`

Строки — марки, сортировка по `TURNOVER` убыв.

Колонка	Поле	Drill-through	Tooltip
Марка	MARK_NAME	ЗН по этой марке	—
Время рем., ч	AVG_SERVICE_TIME		Среднее время ремонта по марке
Выработка, нч	LABOR_TIME		Суммарные нормо-часы
Т-фактор	T_FACTOR		Загрузка мощностей
Потери, мин	WASTED_TIME_MIN		Простой на ЗН
Аккуратность, %	ACCURACY		Качество исполнения

6.2. Экран «Механики» — маршрут `/mechanics` — `src/pages/mechanics/`

Данные: `dashboardStore.mechanicsFiltered` (уже отфильтровано `MECHANIC_ID <> -1`).

⚠ В ответе API нет `mechanicsPrev` — если нужна Δ к прошлому месяцу для механиков, источника для неё сейчас нет. Реализуйте первой версией без Δ, пометьте `TODO`, сообщите лиду.

KPI (4 шт.) — агрегаты по механикам, считаются на фронте

Метрика	Как считать	Tooltip
Механиков	<code>mechanicsFiltered.length</code> (либо <code>MECHANIC_COUNT</code> из <code>kpi</code> — уточнить у лида)	Число механиков с ЗН
Суммарный оборот	<code>SUM(TURNOVER)</code> по <code>mechanicsFiltered</code>	Оборот по всем механикам
Среднее ЗН	<code>AVG(ACCOUNT_COUNT)</code> по <code>mechanicsFiltered</code>	Среднее число ЗН на механика
Средняя аккуратность	<code>AVG(ACCURACY)</code> , игнорируя <code>null</code>	Средняя по механикам

Графики (2 шт.)

График	Тип	Поля	Drill-through	Tooltip
Оборот по механикам	гориз. столбчатый	X=TURNOVER, Y=MECHANIC_NAME	ЗН механика	Механик: оборот Р
Потери vs Аккуратность	точечный (scatter)	X=WASTED_TIME_MIN, Y=ACCURACY, точка=механик	ЗН механика	Оптимум — левый верх

Scatter в Recharts — `ScatterChart + Scatter/ZAxis/Tooltip`. У части механиков `T_FACTOR/ACCURACY` могут быть `null` — такие точки исключить из scatter, но не удалять запись из таблицы ниже.

Таблица «Полная таблица механиков»

Колонки: ACCOUNT_COUNT, TURNOVER, AVG_CASH, ARTICLE_WORK_RATIO, AVG_SERVICE_TIME, LABOR_TIME, T_FACTOR (+Δ — см. предупреждение выше), WASTED_TIME_MIN, ACCURACY.

- Клик по строке → drill-through: `getAccountList(periodYm, dbIndex, { filterString: 'MECHANIC_ID=?', filterParam: [mechanicId] })`
- Tooltip ячеек — как в таблице «Эффективность по маркам» выше

6.3. Порядок работы (рекомендация)

- 1. «Эффективность» — паттерны похожи на «Обзор», меньше нового
- 2. «Механики» — scatter-график и несогласованная Δ, оставить на вторую очередь